Practical 5

Part A:

Aim:

1. Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle, Triangle. Put a mix in a Shape[] and loop once to print each area and the running total, tracking the largest — one loop handling every shape through polymorphism.

Code:

ShapeAreas.java

```
public class ShapeAreas {

    // Abstract parent class
    static abstract class Shape {

        // Abstract method
        public abstract double area();
    }

    // Circle
    static class Circle extends Shape {

        private double radius;

        public Circle(double radius) {
            this.radius = radius;
        }

        @Override
        public double area() {
            return Math.PI * radius * radius;
        }
    }

    // Rectangle
    static class Rectangle extends Shape {

        private double length;
        private double width;

        public Rectangle(double length, double width) {
            this.length = length;
```

```
            this.width = width;
        }

        @Override
        public double area() {
            return length * width;
        }
    }

    // Triangle
    static class Triangle extends Shape {

        private double base;
        private double height;

        public Triangle(double base, double height) {
            this.base = base;
            this.height = height;
        }

        @Override
        public double area() {
            return 0.5 * base * height;
        }
    }

    public static void main(String[] args) {

        // Mixed array of different shapes
        Shape[] shapes = {
            new Circle(5),
            new Rectangle(10, 4),
            new Triangle(6, 8),
            new Circle(3)
        };
```

```
        double total = 0;
        double largest = 0;

        // One loop handles all shapes
        for (Shape shape : shapes) {

            double currentArea = shape.area();

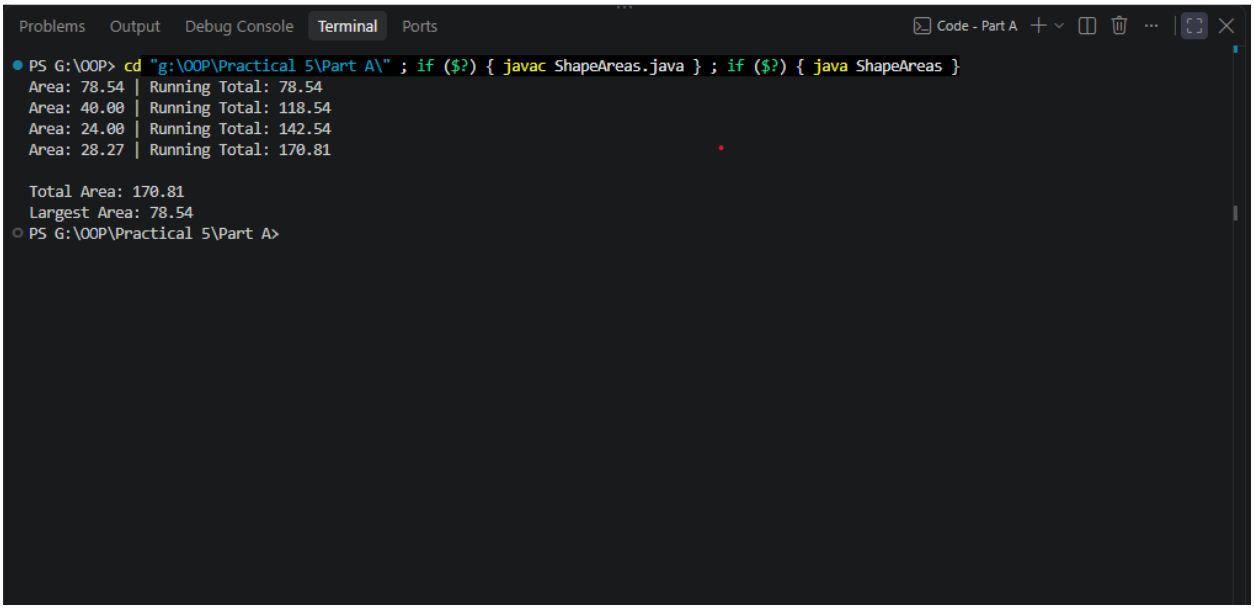
            total += currentArea;

            if (currentArea > largest) {
                largest = currentArea;
            }

            System.out.printf(
                "Area: %.2f | Running Total: %.2f%n",
                currentArea,
                total
            );
        }

        System.out.printf("%nTotal Area: %.2f%n", total);
        System.out.printf("Largest Area: %.2f%n", largest);
    }
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
Code - Part A
PS G:\OOP> cd "g:\OOP\Practical 5\Part A\" ; if ($?) { javac ShapeAreas.java } ; if ($?) { java ShapeAreas }
Area: 78.54 | Running Total: 78.54
Area: 40.00 | Running Total: 118.54
Area: 24.00 | Running Total: 142.54
Area: 28.27 | Running Total: 170.81
Total Area: 170.81
Largest Area: 78.54
PS G:\OOP\Practical 5\Part A>

2. Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns.

Code:

Payroll.java

```
public class Payroll {

    // Abstract Employee class
    static abstract class Employee {

        protected String name;
        protected String id;

        // Constructor
        public Employee(String name, String id) {
            this.name = name;
            this.id = id;
        }

        // Abstract salary method
        public abstract double monthlySalary();

        public String getName() {
            return name;
        }

        public String getId() {
            return id;
        }
    }

    // Full-time employee
    static class FullTime extends Employee {

        private double fixedSalary;
```

```
        public FullTime(String name, String id, double
fixedSalary) {
            super(name, id);
            this.fixedSalary = fixedSalary;
        }

        @Override
        public double monthlySalary() {
            return fixedSalary;
        }
    }

    // Part-time employee
    static class PartTime extends Employee {

        private int hours;
        private double rate;

        public PartTime(
                String name,
                String id,
                int hours,
                double rate) {

            super(name, id);
            this.hours = hours;
            this.rate = rate;
        }

        @Override
        public double monthlySalary() {
            return hours * rate;
        }
    }
```

```
    // Intern
    static class Intern extends Employee {

        private double stipend;

        public Intern(String name, String id, double stipend) {
            super(name, id);
            this.stipend = stipend;
        }

        @Override
        public double monthlySalary() {
            return stipend;
        }
    }

    public static void main(String[] args) {

        Employee[] employees = {
            new FullTime("Rahul", "E001", 50000),
            new PartTime("Priya", "E002", 80, 500),
            new Intern("Amit", "E003", 15000),
            new FullTime("Sara", "E004", 60000)
        };

        double totalSalary = 0;

        for (Employee employee : employees) {

            double salary = employee.monthlySalary();

            totalSalary += salary;

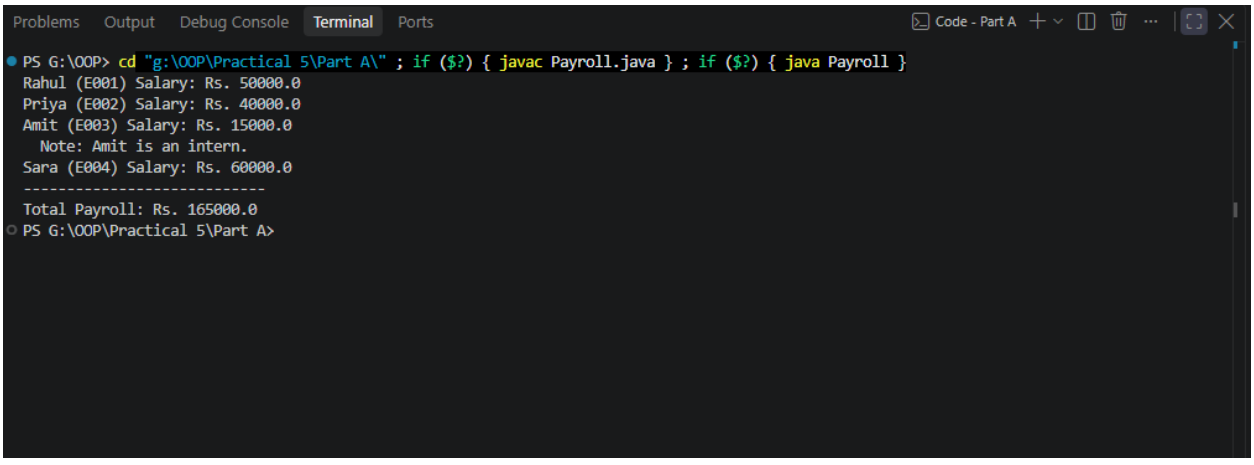
            System.out.println(
                employee.getName()
```

```
                + " (" + employee.getId() + ")"
                + " Salary: Rs. "
                + salary
            );

            // Pattern instanceof
            if (employee instanceof Intern intern) {
                System.out.println(
                    "  Note: " + intern.getName()
                    + " is an intern."
                );
            }
        }

        System.out.println("----------------------------");
        System.out.println("Total Payroll: Rs. " +
totalSalary);
    }
}
```

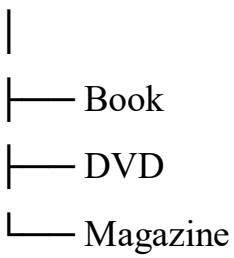
Output



3. (For Advanced Learners – home) Model a few media types that each compute a late fee differently and total the fees for a returned batch. Design the hierarchy and rules yourself.

Design

Media (abstract)



Each subclass overrides `lateFee(int lateDays)`.

Fee Rules

Media Type	Late Fee Rule
Book	₹2 per late day
DVD	₹5 per late day
Magazine	₹1 per late day

We'll store all media in a `Media[]` array and calculate the **total late fee** using runtime polymorphism.

Code:

Media.java

```java
public abstract class Media {

    private String title;

    public Media(String title) {
        this.title = title;
    }

    public String getTitle() {
        return title;
    }

    // Each media type calculates its own fee
    public abstract double lateFee(int lateDays);

    @Override
    public String toString() {
        return title;
    }
}
```

Book.java

```java
public class Book extends Media {

    public Book(String title) {
        super(title);
    }

    @Override
    public double lateFee(int lateDays) {
        return lateDays * 2.0;
    }
}
```

DVD.java

```
public class DVD extends Media {

    public DVD(String title) {
        super(title);
    }

    @Override
    public double lateFee(int lateDays) {
        return lateDays * 5.0;
    }
}
```

Magazine.java

```
public class Magazine extends Media {

    public Magazine(String title) {
        super(title);
    }

    @Override
    public double lateFee(int lateDays) {
        return lateDays * 1.0;
    }
}
```

MediaDemo.java

```
public class MediaDemo {

    public static void main(String[] args) {

        Media[] returnedMedia = {
            new Book("Java Programming"),
            new DVD("Inception"),
            new Magazine("Technology Today"),
            new Book("Data Structures"),
            new DVD("Interstellar")
        };

        int[] lateDays = {
            3,
            2,
            5,
            4,
            1
        };

        double totalFee = 0;

        System.out.println("=== Returned Media ===");

        for (int i = 0; i < returnedMedia.length; i++) {

            double fee = returnedMedia[i].lateFee(lateDays[i]);

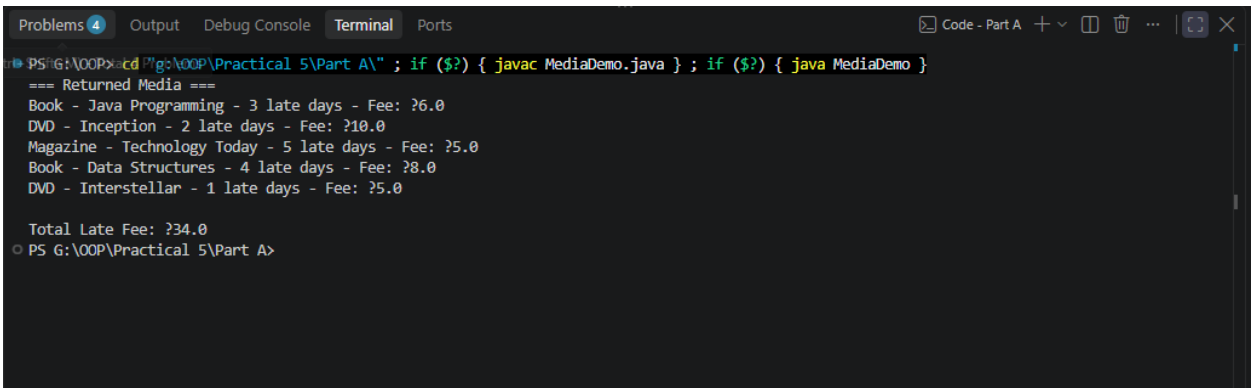
            System.out.println(
                returnedMedia[i].getClass().getSimpleName()
                + " - "
                + returnedMedia[i].getTitle()
                + " - "
                + lateDays[i]
```

```
                + " late days - Fee: ₹"
                + fee
            );

            totalFee += fee;
        }

        System.out.println("\nTotal Late Fee: ₹" + totalFee);
    }
}
```

Output



Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What is the difference between an abstract class and a concrete class?

   Ans.

Abstract Class	Concrete Class
Declared using `abstract` keyword.	Normal class without `abstract`.
Cannot be instantiated directly.	**Can be instantiated** using `new`.
Can contain abstract methods (methods without a body).	Must provide implementations for inherited abstract methods.
Used to define a common structure/behavior for subclasses.	Represents a complete, usable implementation.
Example: `abstract class Account`	Example: `class SavingsAccount extends Account`

2. What is dynamic method dispatch (run-time polymorphism)?

   Ans.

Dynamic method dispatch is the mechanism by which Java decides **at run time** which overridden method should be executed based on the **actual object**, not the reference type.

It is the basis of **run-time polymorphism**.

Example:

```
Account a;

a = new SavingsAccount("Rahul", 10000, 2000);

System.out.println(a.interestRate());
```

3. What does the super keyword do in a subclass constructor?

Ans.

The `super` keyword is used to access members of the **parent class**. In a subclass constructor, `super(...)` specifically calls the **parent class constructor**.

Example:

```
class Account {
    Account(String ownerName, long balance) {
        // initialize account
    }
}

class SavingsAccount extends Account {
    SavingsAccount(String ownerName, long balance, long
minBalance) {
        super(ownerName, balance);
        this.minBalance = minBalance;
    }
}
```